end of the spectrum, so it can be difficult or take a lot of time and energy to establish this tracing-centered (as opposed to logs-centered) approach.

2.3.1 Instrumentation

Let's now see logs in action. For this, we instrument our `echo` service so that it emits log events. The following listing shows our `echo` service instrumented with logs.

Listing 2.3 Example Go program, instrumented to emit structured logs

```
package main

import (
    "fmt"
    "math/rand"
    "net/http"
    "time"

    log "github.com/sirupsen/logrus"
)

func handleEcho(w http.ResponseWriter, r *http.Request) {
    message := r.URL.Query().Get("message")
    log.WithFields(log.Fields{
        "service": "echo",
    }).Info("Got input ", message)
    if rand.Intn(100) > 90 {
        log.WithFields(log.Fields{
            "service": "echo",
        }).Error("Something really bad happened :(")
        w.WriteHeader(500)
        return
    }
    fmt.Fprintf(w, message)
}

func main() {
    rand.Seed(time.Now().UnixNano())
    log.SetFormatter(&log.JSONFormatter{})
    http.HandleFunc("/echo", handleEcho)
    log.Fatal(http.ListenAndServe(":4242", nil))
}
```

No matter what, always emit an INFO log line here that confirms the input value we got via the query parameter message. Note the "service": "echo" in the line above (and again in the error case below): this provides a context for the event, allowing you to pinpoint the service.

Emit an ERROR log line in error case.

Configure the logger to emit log lines formatted in JSON. Also, note that the logger we're using here, logrus (https://github.com/Sirupsen/ logrus), defaults to output to stderr, something we have to consider when we consume the logs, later.

To build the service, you also need to define its dependencies via a file called `go.mod` (shown in the following listing).

Listing 2.4 Example Go program dependencies file `go.mod`

```
module co11yia/ch2/logs

go 1.20

require github.com/sirupsen/logrus v1.8.1
```